GitHub スター上位レポート

WebLLM

2026-11-09 作成 ／ 対象: 過去 7 日（2026-11-02 以降）に作成されたリポジトリ
テーマ: WebLLM

#	区分	リポジトリ	★	一言で
1	テーマ	rushikeshg25/locai	1	WebLLMとWebGPUでブラウザ内だけで動く、アカウント不要のローカルAIチャット・文章作成アシスタント
2	テーマ	AHMADALIPARR/sovereign-jules	1	Google Jules型の非同期コーディングエージェントを、自前環境で動かせるようRust標準ライブラリのみで再実装したデーモン
3	テーマ	alenpepaxx1/AELVOR	1	WebGPU/CPUでブラウザ内にLLMを動かす、英独対応のAIチャット兼サイバーセキュリティ作業環境
4	テーマ	mrducnguyen/webllmchat	0	WebGPUを使い、自分のGPU上でブラウザ内完結のLLMチャットを行うSvelteKit製の静的Webアプリ
5	テーマ	williamcaban/webllm-evalhub	0	EvalHub MCPのdesign_collectionスキルを使い、評価用ベンチマークコレクションを対話で設計するブラウザ向けチャットUI
6	総合	KKKKhazix/agent-autoloop	4,137	Claude CodeやCodexなどのCLIエージェントを常駐で動かし、判断が必要な時だけ通知カードで人に確認する自動化ハブ
7	総合	Minireel/HGhub	3,543	短劇（ショートドラマ）配信サービス「黄果短剧」の非公式サードパーティ製クライアント。Rust と Tauri 2 で作られています。
8	総合	worldonlyu/Google_AI_Pro_2Year_ZeroCost_Auto	3,271	Googleアカウントを自動操作してGoogle AI Proの無料特典を取得すると称するPython製自動化ツール（要注意）
9	総合	NotRexed/Hookshot	3,235	Switch 2版『ゼルダの伝説 時のオカリナ』を静的再コンパイルし、エミュレータなしでWindows/Linux上でネイティブ実行するプロジェクト
10	総合	govijr/terminal-mac	2,401	Apple silicon上で仮想macOSを動かし、その画面をターミナルに表示・操作できるようにするSwift製ツール

テーマ 1. rushikeshg25/locai

https://github.com/rushikeshg25/locai

★ 1 ／ フォーク 0 ／ 言語 TypeScript ／ 作成 2026-11-04
該当テーマ: WebLLM

WebLLMとWebGPUでブラウザ内だけで動く、アカウント不要のローカルAIチャット・文章作成アシスタント

概要

Locaiは、WebLLMを介してWebGPU上で言語モデルをブラウザ内で直接動かすAIチャットアプリです。アカウントやAPIキー、推論サーバーは不要で、メッセージや添付ファイルはローカルで処理されます。会話の保存・検索・ピン留め、分岐付きの再送信、選択テキストの引用、Qwen 3の思考過程の分離表示など、チャットUIとしての機能も一通り備えています。プライバシーを重視しつつ、手元のGPUで小型LLMを手軽に試したい人に向いています。

要点

- WebLLMでLLMをブラウザ内のWebGPU上で実行
- APIキー・アカウント・推論サーバーが一切不要
- 会話や添付ファイル、モデルはブラウザストレージに保存
- Qwen 2.5 0.5BからPhi 3.5 Miniまで7モデルを選択可能
- Web検索はせず、デスクトップ専用でモバイル非対応

何をするものか

Locaiは「ブラウザの中で、自分の端末上で動くAIチャット」です。質問への回答、アイデア出し、文章の改善などをローカルの言語モデルで行います。推論はWebLLMを通じてWebGPUで実行されるため、サーバー側でGPUを用意する必要がありません。独立した執筆モードはなく、一つのチャット画面で質問も文章作成もこなす設計で、アプリ側のペルソナやシステムプロンプトは付けずに会話履歴だけを送ります。なお、Webページの取得や検索は行わず、外部情報は貼り付けるかテキストファイルを添付して与える方式です。

主な機能

会話は最初のメッセージ後に自動保存され、タイトル自動生成、検索、名前変更、ピン留め、削除ができます。起動時は常に新規チャットで始まり、過去の会話はサイドバーに並びます。リトライや編集して再送する際は元の会話を上書きせず分岐させられます。メッセージの一部を選択して引用するSelect-to-cite、最大3つのtxt/mdファイル添付（合計24,000バイト）、Markdown表示とコードのコピー、ライト/ダークテーマ、Markdownエクスポート、一時チャットモードにも対応しています。Qwen 3 0.6BではReasoningトグルで思考部分を折りたたみ表示し、最終回答のみを履歴に再利用します。

仕組み・アーキテクチャ

フロントエンドはReactで構成され、推論はinference.worker.tsというWeb Worker内で@mlc-ai/web-llmを用いて実行されます。モデルの重みはブラウザキャッシュに、会話・添付・テーマ・選択モデルはブラウザのローカルストレージに保存され、一時チャットや未送信の下書きはタブのメモリ上のみに置かれます。本番ビルドではvite-plugin-pwaによるService Workerでアプリ本体をキャッシュします。重みはダウンロード済みでも、訪問ごとにGPUメモリへの準備が必要です。入力はバイト単位の保守的な予算で管理し、古い会話ターンを除外して収め、最新リクエストが大きすぎる場合は黙って切り詰めずに拒否します。保存された会話はLocai側では暗号化されません。

技術構成

言語はTypeScript、UIはReact 19、ビルドツールはVite、Markdown描画にreact-markdownとremark-gfm、差分表示にdiffを使用しています。テストはVitestとTesting Library（jsdom環境）による単体テストと、Playwrightによるe2eテストの二層構成です。e2eの既定では制御されたワーカーのフィクスチャを使ってアプリの挙動を検証し、RUN_MODEL=1を付けると実モデルをダウンロードして試すオプトインのテストも走らせられます。配布用にはnode:24-alpineでビルドし、nginx:stable-alpineで静的配信するマルチステージのDockerfileが用意され、GitHub Actionsのワークフローも含まれます。

使い方・導入方法

Dockerの場合はリポジトリをcloneし、docker compose up --build -d --wait を実行して http://localhost:4174 を開きます。ポートは PORT=8080 のように変更可能です。コンテナにGPUアクセスは不要で、推論はブラウザ側で行われます。Node.jsの場合はNode.js 24とnpmで npm ci、npm run dev を実行し、本番ビルドは npm run build の後に npm run preview を使います。利用時は送信ボタン横のモデルセレクターからモデルを選ぶと、必要に応じてダウンロードして読み込みます。WebGPU対応のデスクトップブラウザ、ハードウェアアクセラレーション、HTTPSまたはlocalhostでのアクセスが前提です。

注目ポイント・注意点

モデル表には重みサイズと推定GPUメモリが記載されており、デフォルトのQwen 2.5 0.5Bは重み280MB、推定945MBと軽量です。最大のPhi 3.5 Miniは推定3.7GBが必要です。IME変換中に送信されない配慮やアクセシブルなラベルなど、日本語入力やキーボード操作への配慮も見られます。注意点として、モバイルは未対応、Web検索なし、アカウント同期なし、モデルの読み込みは10分、生成は2分でタイムアウトします。ストレージはオリジン・ポート・ブラウザプロファイルごとに分かれ、サイトデータ削除で会話も消えます。ライセンスはUNLICENSEDで、利用・改変条件はREADMEからは明示されていません。

技術構成: TypeScript、React 19、Vite、@mlc-ai/web-llm、WebGPU、react-markdown、remark-gfm、diff、vite-plugin-pwa、Vitest、Testing Library、Playwright、Docker、nginx、Node.js 24
向いている人: API課金や外部送信なしでLLMチャットを使いたいプライバシー重視のユーザー、WebLLMやWebGPUによるブラウザ内推論の実装例を学びたいフロントエンドエンジニアや学生、Reactでのチャットアプリ設計やWeb Worker・PWA・Playwrightテストの構成を参考にしたい開発者に向いています。

テーマ 2. AHMADALIPARR/sovereign-jules

https://github.com/AHMADALIPARR/sovereign-jules

★ 1 ／ フォーク 0 ／ 言語 Rust ／ 作成 2026-11-08 ／ NOASSERTION
該当テーマ: WebLLM

Google Jules型の非同期コーディングエージェントを、自前環境で動かせるようRust標準ライブラリのみで再実装したデーモン

概要

Sovereign Julesは、GitHubのissueコメントでボットにメンションするとリポジトリを一時サンドボックスにクローンし、モデルに変更を生成させ、テストを通過したらプルリクエストを開く非同期コーディングエージェントです。README によれば、Google Jules の提供終了を受けて、そのアーキテクチャ（Webhookトリガー、一時サンドボックス、テストゲート、PR作成）を運用者自身のハードウェア上で維持することを目的としています。外部クレートを使わずRust標準ライブラリのみで書かれており、各段階をハッシュチェーン化した追記専用の実行台帳に記録します。ツールを全部プロンプトに詰め込むのではなく登録済みレジストリから候補を探す仕組みや、WebGPU/WebLLMによるローカル推論にも対応している点が特徴です。

要点

- Rust標準ライブラリのみ、外部クレートゼロで実装されたデーモン
- Cloudflare Workerで署名検証とペイロード削減を行ってから転送
- 全段階を改ざん検知可能なハッシュチェーン台帳に記録
- OpenAI互換HTTPかWebGPU上のWebLLMをモデルとして選択可能
- 独自のSnapKitty Opaqueライセンスで商用利用には別途許可が必要

何をするものか

Google Labs の Jules は、リポジトリをクラウドVMにクローンし、バグ修正や機能追加などのタスクを自律的にこなしてプルリクエストを開く非同期コーディングエージェントでした。README では Jules が2026年に提供終了を発表したと説明されており、本リポジトリはその製品ではなく「パターン」を自前の環境で再現することを目的としています。具体的には、GitHub App の Webhook を受けて、タスクごとの一時作業ディレクトリでモデルにコードを書かせ、テストを通過した場合のみブランチをpushしてPRを作成します。クラウドサービスの複製ではなく、同じループを運用者の管理下で動かすことが狙いだと明記されています。

主な機能と処理の流れ

処理は7段階のループで構成されます。まず GitHub App（Contents、Pull Requests、Issues の読み書き権限、issue_comment と push イベント）が Webhook をエッジの Worker に送ります。次に @<bot_name> を含むコメントがトリガーとなり、デーモンがリポジトリ、ブランチ、プロンプトを抽出します。リポジトリはタスク専用の一時ディレクトリにクローンされ、Tool Discovery Engine の封印済みレジストリとプロンプトを照合して候補ツールだけをモデルに渡します。モデルは FILE: ブロックで全ファイル単位の変更を返し、危険なパスは拒否されます。設定したテストコマンドが失敗すると push せず台帳に記録し、成功時は

sovereign/<task-id> ブランチを push してチェックリスト形式の本文でPRを開きます。GitHubトークンがなければドライランとなり、ペイロードは実行ディレクトリに書き出されます。

Webhookとセキュリティの仕組み

GitHub はデーモンに直接通信せず、まず edge/worker.js（Cloudflare Worker）が受け取ります。Worker は X-Hub-Signature-256 を WEBHOOK_SECRET で HMAC-SHA256 検証し、不正なら401で拒否して転送しません。正当な場合はペイロードを action、ref、repository.full_name、clone_url、default_branch、issue.number、issue.title、comment.body、comment.user.login などの必要最小限に削り、インストール情報や sender オブジェクトはエッジ側で破棄します。その後 DAEMON_SECRET で再署名してデーモン（DAEMON_URL）へ送り、デーモン側でも再度HMACを検証します。GitHubトークンは環境変数からのみ読み込まれ、ディスクや台帳には書かれないと説明されています。

モデルバックエンドとダッシュボード

モデルは model_backend という一つの切り替え点で2種類から選べます。http は OpenAI 形式のエンドポイント（ローカルの Ollama やAPI）を使うもの、webllm はデーモンが GET /infer で提供するページをヘッドレス Chromium で開き、WebLLM が WebGPU 経由でVRAM上でモデルを動かすものです。後者は Python、CUDA、外部APIが不要とされ、デーモンは /infer/next と /infer/result でジョブと結果をやり取りします。どちらでも後段の適用・テスト・PR処理は共通です。また GET / では実行台帳をタスクカードとして表示するダッシュボードを提供し、フレームワークもスクリプトも使わない素のCSSで Jules 風のダークテーマを再現しています。

技術構成と導入方法

本体は src/main.rs に集約された Rust 製デーモンで、Cargo.toml に依存クレートの記載はありません。エッジ部分は JavaScript の Cloudflare Worker（edge/worker.js、設定例は edge/wrangler.toml.example）、WebLLM 用のページは webllm/infer.html です。テストには Python 製のスタブモデルや疑似ブラウザ、Node 用の Worker ハーネスが用意されています。ビルドは cargo build --release、テストは ./test/run-tests.sh で実行し、README では18件すべて成功と記載されています。設定は sovereign-jules.conf.example を参考に key = value 形式でポート、ボット名、作業ディレクトリ、モデルのエンドポイント、テストコマンド、git の identity、Webhookシークレット、トークンを保持する環境変数名などを指定します。台帳の整合性は --verify-ledger で再計算して確認できます。

注目ポイントと注意点

外部クレートを一切使わない方針や、ハッシュチェーンで改ざんを検出できる追記専用台帳、エッジでの情報最小化など、自己ホストと監査性を重視した設計が特徴です。一方で STATUS.md の記述によれば、実際の GitHub App 連携ループには所有者のトークンとエッジのルート設定が必要で、WebGPU推論にはGPU搭載マシンが必要という既知の制約があります。WebGPUの経路はテストではスタンドインのクライアントで検証されたとされています。サンドボックスの隔離の強度（コンテナ等を使うか）は README からは不明です。ライセンスは独自の SnapKitty Opaque Source License で、研究・学習・内部評価・セキュリティ分析は許可されていますが商用利用には別途書面での許可が必要です。スター数も少なく、作成直後の初期段階のプロジェクトである点にも留意が必要です。

技術構成: Rust（標準ライブラリのみ）、JavaScript（Cloudflare Worker）、Cloudflare Workers / wrangler、WebLLM、WebGPU、ヘッドレス Chromium、Ollama（OpenAI互換エンドポイント）、GitHub App / Webhook、HMAC-SHA256、Python・Node.js（テスト用スタブ）
向いている人: Google Jules のような非同期コーディングエージェントを自前のサーバーやGPUマシンで運用したい開発者、外部依存を最小にしたRust実装やハッシュチェーン台帳・Webhook検証の設計を学びたいエンジニアや学生、AIエージェントのセキュリティや監査性を研究・評価したい人に向いています。ライセンス上、商用での利用には別途許可が必要です。

テーマ 3. alenpepaxx1/AELVOR

https://github.com/alenpepaxx1/AELVOR

★ 1 ／ フォーク 0 ／ 言語 TypeScript ／ 作成 2026-11-07 ／ MIT
該当テーマ: WebLLM

WebGPU/CPUでブラウザ内にLLMを動かす、英独対応のAIチャット兼サイバーセキュリティ作業環境

概要

AELVORは、外部のAI APIや推論サービスを使わず、WebLLMで選んだQwenなどの事前学習済みモデルをブラウザ上でローカル実行するAIチャットワークスペースです。英語とドイツ語の会話、音声入出力、ナレッジライブラリからの抜粋参照、コード生成とサンドボックスでのWebプレビュー、コードプロジェクト管理などを1つの画面にまとめています。モデルを使わない決定的な「Local tools」モードや、自前で学習できるTF-IDFベースのサイバーセキュリティ分類器も同梱しています。READMEは各機能の制約やテスト範囲、未検証の部分を非常に細かく明記しているのが特徴です。

要点

- WebLLM 0.2.86でQwen等159のテキストモデルをブラウザ内実行
- AI APIキー不要、会話やデータはブラウザのストレージに保存
- 生成コードをsandbox iframeでプレビュー、プロジェクト管理やZIP出力も可能
- LLMなしで動くLocal toolsと学習可能なセキュリティ分類器を同梱
- Node.js製ブリッジ経由でWindowsのPowerShell/CMDを確認付きで実行

何をするものか

AELVORは、ブラウザだけで完結するAIアシスタント兼セキュリティ作業スペースです。AI Chatモードでは、WebLLMのカタログからQwen 1.5B（既定）、0.5B、3Bなどを選び、明示的に「Download & load selected model」を押した時点でHugging Faceから重みを取得し、以降の推論はローカルで行います。外部のモデル推論サービスやAPIキーは使いません。会話は英語とドイツ語に対応し、ストリーミング表示、停止、続きの生成、ブラウザ音声による読み上げと音声入力が使えます。READMEは、これらが事前学習済みモデルであり、すべての質問に正しく答えられるわけではなく、Web閲覧やコード実行もしないと明記しています。

主な機能

会話機能として、プロジェクト分け、ピン留め、検索（Ctrl/Cmd+K）、分岐による質問の編集や再回答、Refine（簡潔化、例示、独英翻訳など）、下書きの自動保存、JSONバックアップの入出力があります。ナレッジライブラリにはTXTやMarkdown、CSV、JSONなどを登録でき、質問ごとに語彙ベースのランキングで最大3件の抜粋を選び、[K1]〜[K3]として引用させます。許可リストにある公開ドキュメントのURLを読み込む機能もあります。コード面では、HTML/CSS/JS/React(TSX)のプレビュー、Web・React・Python・Node.js・Android・iOS・Flutterのテンプレートを持つコードプロジェクト、差分レビュー、バージョン履歴、GitHub公開リポジトリやZIPの取り込みを備えます。

仕組み・アーキテクチャ

WebLLMは専用のWeb Workerで動作し、UIの応答性を保ちます。読み込み前には毎回WebGPUの有無やバッファ上限などのデバイスチェックを行い、モデルのメタデータやキャッシュ済み重みのサイズも検証します。最後に90秒以内の短い応答テストに成功した場合だけReadyと表示します。WebGPUを使わないCPU専用のWebAssembly SIMDバックエンドも用意されています。プロンプトは直近最大4往復と現在の質問を6,000文字の予算に収め、出力は最大768トークンに制限されます。プレビューするユーザーコードは sandbox="allow-scripts" のiframeと制限的なCSPの中で実行され、TSXのトランスパイルは別のWorkerで行います。

Local toolsとセキュリティ分類器

Local toolsは言語モデルをダウンロードせずに動く決定的なエージェントです。URLの構造チェック（実際にはアクセスしない）、SQLインジェクションや動的実行などを行単位のパターンで見つけるコードレビュー、ログ中の認証失敗やIPv4候補の集計、3種類の固定Pythonスクリプトの生成、上限付きの厳密な四則演算に対応します。別枠の分類器はTF-IDFによるsoftmaxとNaive Bayesを組み合わせたHybrid v2で、Web Worker内のSGDで学習します。96件の英独サンプルと8つのトピックで学習済みの重みが public/models に入っています。3分割交差検証による指標を表示し、Python標準ライブラリだけで動く推論スクリプトも出力できます。

技術構成と導入方法

TypeScriptで書かれ、Next.js 16とReact 19を使い、vinext、Vite 8、Cloudflareのwrangler環境でビルド・実行する構成です。UIはshadcn系コンポーネント、Radix UI、Tailwind CSS 4で作られています。package.jsonでは Node.js 22.13.0以上、pnpmが指定されており、scriptsに dev、build、start（wrangler dev）、lint などがあります。READMEに手順として明記されたインストール方法は見当たらず、START-HERE.md の内容も今回の資料からは不明です。ワークスペースはSitesのサインインでオーナー専用にすると書かれていますが、詳しいホスティング手順はREADMEからは不明です。AI Chatの利用には、WebGPU対応のブラウザとGPUメモリ（目安1.0〜2.9GB）が必要です。

注目ポイント・注意点

READMEは、できることとできないことを非常に率直に書いている点が目を引きます。たとえば「自己レビューは独立した検証ではない」「引用の正しさは保証しない」「実機のタッチ操作やGPU推論は未検証」といった記述が多く、テストの範囲と未検証の領域が区別されています。Windowsブリッジはローカルの Node.jsスクリプトで、127.0.0.1にのみバインドし、セッショントークンとユーザーによる確認を経て1コマンドずつ実行します。ただしファイルシステムのサンドボックスではない点に注意が必要です。ライセンスはメタデータではMITですが、package.jsonでは UNLICENSED になっており、COPYRIGHTファイルも存在するため、利用前に確認することをおすすめします。スター1件の個人プロジェクトです。

技術構成: TypeScript、Next.js 16、React 19、vinext、Vite 8、Cloudflare wrangler / @cloudflare/vite-plugin、@mlc-ai/web-llm 0.2.86、@huggingface/transformers 3.8.1、WebGPU、Web Workers、Tailwind CSS 4、shadcn / Radix UI、drizzle-orm、zod、lucide-react、Node.js 22（Windowsブリッジ）、Python（分類器推論ランナー）
向いている人: APIキーを使わず、プライバシーを保ったままブラウザ内でLLMを試したい開発者や学生に向いています。WebLLMやWebGPUによるオンデバイス推論の実装例、サンドボックス化したコードプレビューや堅牢なブラウ

ザストレージ処理の設計を学びたい人にも参考になります。英独バイリンガルのAIワークスペースや、小規模なセキュリティ分類器の学習・評価に関心がある人にも適しています。

テーマ 4. mrducnguyen/webllmchat

https://github.com/mrducnguyen/webllmchat

★ 0 ／ フォーク 0 ／ 言語 Svelte ／ 作成 2026-11-05

該当テーマ: WebLLM

WebGPUを使い、自分のGPU上でブラウザ内完結のLLMチャットを行うSvelteKit製の静的Webアプリ

概要

WebLLM Chatは、@mlc-ai/web-llmを利用して、ユーザーのGPU上でブラウザだけでAIモデルを動かしチャットできるアプリです。バックエンドや解析ツールを持たず、チャット履歴も保存しないため、外部へ送信されるのはモデル一覧とモデルファイルの取得のみというプライバシー重視の設計です。モデル一覧はHugging Faceのmlc-aiリポジトリとGitHubの公式ライブラリ一覧から日次で更新され、実行コードの取得元はContent Security Policyとworker内のfetchガードで厳格に制限されています。watere.appの一部として公開されているプロジェクトです。

要点

- WebGPUで自分のGPUを使い、ブラウザ内だけでLLMを実行
- バックエンド・解析なし、チャットも保存しない設計
- モデル一覧はHugging FaceとGitHubから日次で自動更新
- CSPとworkerのfetchガードで通信先を厳格に制限
- SvelteKitのadapter-staticで静的サイトとしてビルド

何をするものか

WebLLM Chatは、ブラウザ上でAIモデルと対話するためのチャットアプリです。推論はサーバーではなく、WebGPUを通じて利用者自身のGPUで実行されます。中核には、MLCプロジェクトが開発する@mlc-ai/web-llmが使われています。READMEによれば、ブラウザの外に出る通信はモデル一覧とモデルファイルの取得だけで、バックエンドも解析ツールも存在せず、チャットは保存されません。手元の環境だけで完結するローカルLLMチャットを、インストール不要のWebページとして提供している点が特徴です。watere.appというサイトの一部として位置づけられています。

モデルの取得元と更新の仕組み

モデル一覧は二段構えです。まず、インストール済みの@mlc-ai/web-llmに含まれるprebuiltAppConfigを組み込みリストとして使うため、ネットワークがなくても一覧は表示できます。加えて、各日の初回読み込み時やRefreshボタンを押したときには、Hugging Face上のmlc-aiのリポジトリとGitHubの公式ライブラリ一覧を読み込みます。インストールされているmodelVersionに対応するライブラリが存在する場合だけ、そのリポジトリを一覧に追加します。更新に失敗したときは、最後に正常に取得できたリストを保持し続けます。こうした処理はsrc/lib/models.tsが担っていると考えられ、models.test.tsというテストファイルも用意されています。

セキュリティ設計

実行コードにあたるWASMのモデルライブラリは、web-llmプロジェクト自身のディレクトリであるraw.githubusercontent.com/mlc-ai/binary-mlc-llm-libsからのみ読み込みます。取得したモデル一覧はあくまでデータとして読み込み、コードとして実行することはありません。許可された通信先は2か所で強制されています。1つはページに設定されたContent Security Policyで、svelte.config.jsに定義されています。もう1つはworker内のfetchガードで、src/lib/worker.tsにあります。ファイル構成にはsrc/lib/hosts.tsもあり、許可ホストの定義を集約しているものと見られますが、詳細はREADMEからは不明です。

技術構成とアーキテクチャ

フロントエンドはSvelte 5とSvelteKit 2で書かれ、@sveltejs/adapter-staticによって静的サイトとして出力されます。スタイリングにはTailwind CSS 4、アイコンには@lucide/svelte、ビルドにはVite 8、テストにはVitestを使っています。言語はTypeScriptで、WebGPUの型定義として@webgpu/typesを導入しています。src/lib配下には、推論エンジンを扱うengine.ts、WebGPU対応を判定するとみられるwebgpu.ts、Web Worker用のworker.tsがあります。UIコンポーネントは、ChatView.svelteとModelPicker.svelteに分かれています。推論をWorker側で動かしてUIスレッドを塞がない構成と推測されますが、詳細はREADMEからは不明です。

使い方・導入方法

開発にはpnpmを使います。pnpm installで依存関係を入れ、pnpm devでhttp://localhost:5173に開発サーバーが立ち上がります。型チェックはpnpm check、Lintはpnpm lint、テストはpnpm testで実行できます。pnpm buildを実行すると、build/ディレクトリに静的サイトが出力されます。このサイトは/webllmchat/というパスから配信される前提になっているため、別の場所で公開する場合は設定の調整が必要になる可能性があります。実際に利用するには、WebGPUに対応したブラウザとGPUが必要です。具体的な動作要件はREADMEからは不明です。

注目ポイント・注意点

プライバシーを重視したローカル推論と、通信先をCSPとfetchガードの二重で絞り込む堅実な設計は、ブラウザ内LLMアプリの参考実装として価値があります。一方で、スター数やフォーク数は0で、バージョンも0.1.0と初期段階です。ライセンスも明記されていないため、コードを再利用する際は注意が必要です。また、モデルファイルはサイズが大きくなりがちで、初回の読み込みには時間と通信量がかかることが想定されます。ただし、キャッシュの扱いについてはREADMEからは不明です。チャット履歴は保存されないため、会話を残したい用途には向きません。

技術構成: Svelte 5、SvelteKit、TypeScript、@mlc-ai/web-llm、WebGPU、Vite、Tailwind CSS、@lucide/svelte、@sveltejs/adapter-static、Vitest、ESLint、Prettier、pnpm
向いている人: サーバーにデータを送らずにLLMを試したいプライバシー重視の利用者や、WebGPUとweb-llmを使ったブラウザ内推論アプリの作り方を学びたいフロントエンドエンジニア・学生に向いています。CSPやWorker内のfetchガードによる通信制限の実装例を参考にしたい人にも有用です。

テーマ 5. williamcaban/webllm-evalhub

https://github.com/williamcaban/webllm-evalhub

★ 0 ／ フォーク 0 ／ 言語 TypeScript ／ 作成 2026-11-08
該当テーマ: WebLLM

EvalHub MCPのdesign_collectionスキルを使い、評価用ベンチマークコレクションを対話で設計するブラウザ向けチャットUI

概要

評価の目的を自然言語で伝えると、EvalHub MCPサーバーの design_collection スキルがベンチマークコレクションを提案します。ユーザーはそれを対話で手直しし、create_collection で作成、必要に応じて submit_evaluation で評価を投入できます。UIはPatternFly 6とReactで作られたブラウザアプリで、クラスタの認証情報はローカルのPythonプロキシが保持し、ページ側には渡りません。モデルにはOllamaなどのOpenAI互換エンドポイントのほか、推論サーバーなしでWebGPUを使いタブ内で動く小型モデルも選べます。

要点

- EvalHub MCPで評価コレクションを対話的に設計・作成するチャットUI
- PatternFly 6 + React + Viteで構築され、ブラウザ上で動作
- cors-proxy.pyがトークンを保持し、CORS制約も回避する
- LLMはOllama等のOpenAI互換APIか、WebGPUのブラウザ内モデル
- OpenShift上のラボ・開発環境向けで、既知の制約や回避策が詳しい

何をするものか

EvalHub の MCP サーバーが提供する design_collection スキルを、チャット形式で使うためのフロントエンドです。ユーザーが評価したい内容を説明すると、モデルが MCP ツールを呼び出してベンチマークコレクションの案を出します。ユーザーはそれを繰り返し調整し、最後に create_collection でコレクションを作成し、必要なら submit_evaluation で評価を実行に回します。作成したコレクションは OpenShift クラスタ上の EvalHub API とデータベースに保存されます。リポジトリには UI 本体に加えて、ローカルプロキシ、接続用シェルスクリプト、evalhub CLI を動かす Python 環境が含まれています。

仕組み・アーキテクチャ

ブラウザが通信する相手は 127.0.0.1 だけです。チャット補完は Ollama(127.0.0.1:11434)か、タブ内の WebGPU モデルに送られます。MCP 呼び出しは http://127.0.0.1:8787/mcp を経由して cors-proxy.py に届きます。プロキシは Authorization: Bearer トークンと X-Tenant ヘッダーを付け、OpenShift の Route evalhub-mcp に HTTPS で転送します。クラスタ内では kube-rbac-proxy が mcp-client のトークンを検証し、その先の MCP サーバー(:8445)が evalhub-upstream サービスの EvalHub API を呼び出します。プロキシが必要な理由は二つあり、MCP Route が CORS ヘッダーを返さないことと、トークンをページに置かないことです。

主な機能と技術構成

ui/src 以下には React コンポーネントがまとまっています。主なものは、ツール呼び出しを表示する ToolCallCard、コレクションのプレビューを表示する CollectionPreview、推論過程を表示する ReasoningBlock、数式表示用の MathExpr、MCP の接続状態を示す McpStatusPill、各種設定を行う SettingsPanel などです。lib 側には、エージェントループの agent.ts、MCP クライアント、Ollama 連携、ブラウザ内モデル用の engine.ts と worker.ts、ツール呼び出し形式の変換を担う toolFormats.ts などが置かれています。また、MCP に接続できないときに使う Demo モード用のデータもあります。テストファイルも複数含まれています。WebGPU 推論に使っている具体的なライブラリ名は、README からは不明です。Python 側は Python 3.12 と uv で環境を管理し、eval-hub-sdk[cli] を依存に持ちます。

使い方・導入方法

必要なものは Node 20 以上、Python 3.12 以上、クラスタにログイン済みの oc、そしてモデルのエンドポイントです。手順は三つあります。まず cd ui && npm install && npm run build で UI をビルドします。次に ./evalhub-connect.sh -n evalhub-mcp-demo --insecure --long-lived を実行すると、MCP Route を探してトークンを取得し、evalhub-chat.config.json(パーミッション600、git管理外)を書き出します。最後に python3 -I cors-proxy.py --config evalhub-chat.config.json でプロキシを起動し、http://127.0.0.1:8787/ を開きます。--long-lived を付けない場合、トークンの有効期限は2時間です。スクリプトを再実行すればトークンが更新され、プロキシは再起動しなくても新しい設定を読み込みます。

注目ポイント・注意点

--insecure を付けると、プロキシは Route の証明書を検証しなくなります。ラボクラスタでは、ルーター証明書が Route のホスト名と一致しないためにこの指定が必要ですが、信頼できる環境以外では使わないよう README は注意しています。プラットフォームが提供する MCP イメージ(1.0.3)には design_collection がありません。そのため、上流イメージ(1.0.4)と別立ての evalhub-upstream サーバーを組み合わせるオーバーライドで動かしています。evalhub-upstream のコレクションはメモリ上に保存されるので、Pod が再起動すると消えます。また eval-hub-sdk の制約により、category を持たないコレクションがあると evalhub collections list が失敗する既知の問題があります。ライセンスは明記されていません。

技術構成: TypeScript、React、PatternFly 6、Vite、WebGPU、Ollama(OpenAI互換API)、MCP、Python 3.12、uv、eval-hub-sdk[cli]、OpenShift、kube-rbac-proxy
向いている人: OpenShift AI や EvalHub を使って LLM の評価基盤を検証しているエンジニアに向いています。MCP ツールをブラウザのチャットから呼び出す構成や、認証情報をブラウザに置かないプロキシ設計の実例を学びたい人の参考にもなります。一方で、ラボ・開発クラスタを前提とした内容が多く、一般向けにすぐ使えるツールではありません。

総合 6. KKKKhazix/agent-autoloop

https://github.com/KKKKhazix/agent-autoloop

★ 4,137 ／ フォーク 753 ／ 言語 TypeScript ／ 作成 2026-11-03 ／ MIT

Claude CodeやCodexなどのCLIエージェントを常駐で動かし、判断が必要な時だけ通知カードで人に確認する自動化ハブ

概要

Agent Autoloopは、Claude CodeやCodexのようなコマンドラインAgentを「人が端末に張り付いて使う」状態から解放し、バックグラウンドで仕事を受けて最後まで追いかけさせるための軽量な自動化基盤です。定期実行、GitHub Issue、業務システムのエラーなどのイベントをルールに従ってAgentに割り振ります。判断に迷う操作(データ削除、サービス再起動、支払い、外部送信など)の時だけ、飛書・デスクトップ通知・Slackなどにカードを送って人に決めてもらいます。待機中はAgentプロセスを終了させてTokenを消費せず、回答後に元の会話を再開する点や、イベントソーシングで設計された依存ゼロのカーネルが特徴です。

要点

- CLIエージェントを常駐させ、迷った時だけ通知カードで人に確認
- 確認待ちの間はプロセス終了、Tokenを消費しない
- 約1850行・ランタイム依存ゼロのイベントソーシング型カーネル
- イベント源・Agent・通知・デリバリーを全てプラグイン化
- rules.jsonに書くだけで定期実行やIssue対応を設定可能

何をするものか

手元のWindows、macOS、Linuxマシン1台で動く自動化ハブです。クラウドサーバーや公開ポートは不要とされています。流れは5段階です。まずイベントが入り、カーネルがルールに従ってどのAgentにどのディレクトリで作業させるかを決めます。Agentは1ラウンド作業したら最後の返信に結果を書いて終了します。確信が持てない場合は通知カードで人に尋ね、人が選択すると元の会話を引き継いで残りの作業を進めます。自力で片付く作業は、そのまま完了させるか、30分後などに再チェックを予約する自動ループで処理し、人の手を煩わせません。管理画面は持たず、状況確認はコマンドで行います。

主な機能とシナリオテンプレート

ルールは ~/.agent-autoloop/rules.json に書きます。項目は when(cron またはイベント源と種類)、to(Agent名)、cwd、prompt、skill、parallel などです。保存すると数秒で反映されます。agent-autoloop check で検証でき、rules explain で有効なルールと次回実行時刻を確認でき、emit --dry-run で振り分けを予行できます。pause、resume、run でルール単位の停止・再開・即時実行もできます。Agentは返信の末尾に agent-autoloop コードブロックを付けることで、ask(質問)、notice(通知)、resolve(カードを閉じる)、later(再チェック予約)、events(新イベント発行)を指示できます。同梱テンプレートは、毎日の朝報、GitHub Issue追跡、障害アラート調査、ユーザーフィードバック処理、Webサイト巡回、飛書週報、受信箱の仕分け、PR自動デリバリーなどです。

仕組み・アーキテクチャ

カーネルはシステム時計も乱数も読まず、外部へのリクエストも送りません。同じ入力からは常に同じ結果が出る設計です。中核の仕組みは4つあります。1つ目は「待機中ゼロ消費」で、待ち状態はDBに保存され、Agentプロセスは残りません。2つ目は「イベントソーシング」で、全ての状態を追記専用ログから算出し、ログとインデックスを同一トランザクションで永続化してから応答します。3つ目は「命令とデータの分離」で、Issue本文などの外部テキストはファイルとして渡し、プロンプトには埋め込みません。イベントに cwd、agent、access が含まれていれば拒否します。4つ目は「ホップ数ブレーカー」で、Agentが発行したイベントの連鎖が8層を超えると、記録だけ残して実行を止めます。ディレクトリは kernel、host(Node HTTPやSQLite接続)、protocol、worker、cli、plugins に分かれています。

技術構成とプラグイン

TypeScriptで書かれ、Node 24以上で .ts ファイルを直接実行する構成です。package.json の bin は bin/agent-autoloop.ts を指します。devDependencies は typescript、prettier、@types/node のみで、テストは node --test を使います。ストレージにはSQLiteを使うとREADMEに記載されています。プラグインは4方向に分かれます。イベント源は GitHub/GitLab Webhook、HTTP、cron、Issueポーリングスクリプト。Agentは Claude Code、Codex、ACP、DeepSeek Harness、汎用コマンド。通知チャネルはデスクトップ通知、飛書、钉钉、企業微信、Slack、Discord。これに delivery が加わります。プロンプトを受け取って最後の返信を出力できるCLIなら use: "command" で1行接続でき、Gemini CLIやOpenCodeはACP経由で接続できます。MCPからもカードの確認と回答が可能です。

使い方・導入方法

必要なものは、Node 24以上、Git、ログイン済みのCLI Agentです。macOS/Linuxではさらにcurlが必要です。導入は、macOS/Linuxなら curl -fsSL https://raw.githubusercontent.com/KKKKhazix/agent-autoloop/main/install.sh | sh を、Windows PowerShellなら irm https://raw.githubusercontent.com/KKKKhazix/agent-autoloop/main/install.ps1 | iex を実行します。インストール後に agent-autoloop try を実行すると、「質問→終了して待機→回答→続きを実行」の流れを実機で試せます。回答は agent-autoloop answer 1 2、状況確認は agent-autoloop status で行います。status ではラウンド数、Token、費用も表示されます。agent-autoloop why 1 で各ラウンドの経緯を追え、更新は agent-autoloop update です。

注目ポイント・注意点

信頼性の検証に力が入っています。シードで再現できる20万ステップの障害シミュレーション(sim/random.ts)、1000回の強制終了テスト(sim/crash.ts)、変異テストなどが用意されています。M5 Pro環境での実測値は、コールドスタート約89ms、ディスパッチp99が1.82msです。一方、READMEは検証範囲の限界も明記しています。Webhookテンプレート4種はローカルの模擬サーバーでしか検証されていません。GitLabは実環境に未接続で、多くのコマンド接続例も未実行です。業務テンプレートは実アカウントで完走していません。履歴が増えると状態集計が遅くなり、フォーマット変更後の初回起動ではログから全量を再構築します(110万件で43.1秒)。また、高頻度の自動化では個人サブスクリプションではなくAPIキーの利用が推奨されています。

技術構成: TypeScript、Node.js 24+、SQLite、node:test、Prettier、MCP、ACP、GitHub Actions
向いている人: Claude CodeやCodexなどのCLIエージェントを日常的に使っていて、定期レポート、Issue対応、障害調査、PRのマージからデプロイまでを無人で回したい個人開発者や小規模チームに向いています。飛書や钉钉など中国系ツールへの対応が厚い一方、SlackやDiscordにも対応しています。イベントソーシングや決定論的カーネル設計に関心のあるエンジニアが、設計を学ぶ題材としても有用です。

総合 7. Minireel/HGhub

https://github.com/Minireel/HGhub

★ 3,543 ／ フォーク 1,449 ／ 言語 TypeScript ／ 作成 2026-11-03 ／ NOASSERTION

短劇（ショートドラマ）配信サービス「黄果短剧」の非公式サードパーティ製クライアント。Rust と Tauri 2 で作られています。

概要

HGhub は、縦型ショートドラマ配信サイト「黄果短剧」（YellowFruit）を Windows・macOS・Android・iOS で閲覧するための非公式クライアントです。作者が Tauri 2 のモバイル開発を試すために作ったもので、Rust 製バックエンドと React 19 製フロントエンドで、Fluent デザインの UI と没入型の縦画面再生を実現しています。技術的には、上流 API を単一エンドポイントで扱う設計、Rust 側での画像復号、カスタムプロトコル経由の HLS プロキシなど、Tauri アプリの実装例として参考になる点があります。一方で、第三者のコンテンツを扱うため著作権や利用規約上の懸念が大きく、作者自身も上流データ取得部分の保守を停止し、権利者から連絡があれば公開を停止すると明言しています。

要点

- 黄果短剧の非公式クライアントで、作者は学習・技術交流目的と明記
- Tauri 2 + React 19 で Windows/macOS/Android/iOS に対応
- v1.1.0 以降は PolyForm Noncommercial 1.0.0 で商用利用は禁止
- 上流データ取得部分は今後保守せず、UI/UX 改善が中心になる方針
- 有料化された改変版 hichuv/HGhub は非公式なので注意と警告あり

何をするものか

HGhub は、中国の縦型ショートドラマ配信サイト「黄果短剧」の作品を閲覧・再生するための非公式クライアントです。README では、作者が Tauri 2 のモバイル開発を体験するために作った「小さなおもちゃ」と説明されています。公開後に予想外に多くのスターを集めたため、作者は潜在的なリスクを意識し、ライセンスを非商用限定に変更して免責文を追加しました。本プロジェクト自体は音声や映像を提供・保存・配布しないとしています。すべてのコンテンツは第三者サイトのもので、著作権は原作者と上流サイトに帰属すると明記されています。ページ構成は Home、Search、Category、Profile、Posts、Settings で、ドラマ再生のほかに投稿記事（吃瓜正文）の閲覧機能もあります。

主な機能

縦画面で没入できる再生画面（Feed）、検索、カテゴリ一覧、プロフィール、ローカルお気に入り、テーマ切り替えなどを備えています。設定には「起動時にミュート」があり、既定でオンです。オフにすると前回のミュート状態を復元し、音量は常に記憶されます。ファイル構成からは、回線切り替え（LinePicker）、ログイン（LoginModal）、シーク入力、再生診断などのコンポーネントがあることがわかります。ミラー回線はゲスト閲覧のみに対応し、サイトの切り替え時や初回アップデート後は再ログインが必要です。本プロジェクトにダウンロード機能はありません。README は、一括取得や高頻度アクセスのためにコードを改変しないよう明確に求めています。

仕組み・アーキテクチャ

Rust 側は役割ごとに分かれています。net.rs が通信層（エッジ封筒とリトライ）、api.rs が業務コマンド、media.rs がカバー画像の処理・HLS プロキシ・キャッシュ、session.rs がトークンの永続化を担当します。上流 API は「POST /api/edge/request」という単一の入口にまとまっており、クライアントはここだけに Bearer トークン付きで接続します。カバー画像は Rust 側で AES-128-CBC を復号し、カスタムプロトコル経由でフロントに渡します。m3u8 と動画セグメントもカスタムプロトコルで中継し、WebView の CORS 制限と配信元側の制限を回避しています。カスタムプロトコルの基底 URL はプラットフォームで異なり、Windows/Android は http://hg.localhost、macOS/iOS は hg://localhost です。この基底 URL は lib.rs と api.ts で一致させる必要があります。作者は署名の逆解析はしておらず、ブラウザと同じ同一オリジンの挙動を再利用しているだけだと述べています。

技術構成

アプリの外殻は Tauri 2（Rust 1.90 以上）です。フロントエンドは React 19、TypeScript、Vite 7、Tailwind CSS 4 で構成されています。動画再生には hls.js（HLS / AES-128 対応）を使い、通信には reqwest と rustls を使っています。そのほかの依存関係は clsx、tailwind-merge、DOMPurify（本文 HTML のサニタイズ用）、@tauri-apps/api です。開発用には playwright-core が入っており、tests ディレクトリには HTML セキュリティ、API 読み込み、シーク、セキュリティ設定のテストがあります。docs/security.md では、FakeIP や内部ネットワークへのアクセス制限、アカウントと回線の分離などのセキュリティ対策を説明しています。rules ディレクトリには Mihomo / sing-box 向けの通信振り分けルールも含まれています。

使い方・導入方法

配布物の公式な入手先は GitHub Releases だけで、それ以外の配布元は非公式とされています。ソースからビルドする場合は、pnpm install の後に pnpm app:dev で開発、pnpm app:build でパッケージ化します。Windows では NSIS インストーラー、macOS では .app と .dmg が生成されます。Android は Android SDK・NDK と JDK 17 を用意し、pnpm android:init、android:dev、android:build の順に実行します。iOS は Xcode、xcodegen、CocoaPods と Apple 開発者アカウントを使い、自分で署名して自分の端末にインストールします。プロジェクトを exFAT ドライブに置く場合は、.npmrc の node-linker=hoisted を残してください。配布される dmg は未署名なので、macOS では Gatekeeper に止められる場合があります。

注目ポイント・注意点

Tauri 2 によるデスクトップとモバイル両対応や、カスタムプロトコルでのメディア中継の実装例として、技術的に参考になる点はあります。ただし、本アプリは第三者の有料・権利付きコンテンツを非公式に扱います。上流サイトの利用規約や著作権、利用地域の法令に抵触するおそれがあり、責任は利用者自身が負うと README に明記されています。作者は上流データ取得の保守をやめると宣言しており、今後の動作継続も保証されません。ライセンスは v1.1.0 以降 PolyForm Noncommercial 1.0.0 です（v1.0.1 以前は MIT）。商用利用はできず、透かしの除去やコンテンツの再配布も禁止されています。カード認証や VIP 課金を追加した改変版 hichuv/HGhub は非公式で、安全性も確認できないため、使わないよう警告されています。

技術構成: TypeScript、Rust、Tauri 2、React 19、Vite 7、Tailwind CSS 4、hls.js、reqwest、rustls、DOMPurify、clsx、tailwind-merge、playwright-core、pnpm
向いている人: Tauri 2 でデスクトップとモバイル（Android/iOS）両対応のアプリを作る方法を学びたい人に向いて

います。特に、カスタムプロトコルによるメディア中継、HLS 再生、Rust とフロントエンドの役割分担の実例を調べたい開発者や学生にとって、コードを読む教材として役立ちます。一方、コンテンツ視聴を目的に使う場合は、著作権・利用規約・各地の法令上のリスクを自分で判断する必要があり、商用利用はできません。

総合 8. worldonlyu/Google_AI_Pro_2Year_ZeroCost_Auto

https://github.com/worldonlyu/Google_AI_Pro_2Year_ZeroCost_Auto

★ 3,271 ／ フォーク 1,205 ／ 言語 Python ／ 作成 2026-11-04

Googleアカウントを自動操作してGoogle AI Proの無料特典を取得すると称するPython製自動化ツール（要注意）

概要

このリポジトリは、Googleアカウントへ自動ログインし、支払いカードを登録してGoogle AI Proの「2年間無料（US$0.00）」特典を全自動で取得すると謳うツールです。FastAPI製のローカルWeb管理コンソール（http://127.0.0.1:8765）上で、支払いカード情報や複数のGoogleアカウントを一括登録し、Playwrightでブラウザを自動操作します。READMEによれば作者自身の開発ではなく「学習用の共有」とされていますが、他者の決済情報や大量アカウントの自動操作を伴う内容で、重大な規約違反・不正利用・詐欺のリスクを含む点に強く注意が必要です。

要点

- FastAPI+PlaywrightによるローカルGoogle管理コンソールでGoogleを自動操作
- 支払いカード情報と大量のGoogleアカウントを一括登録して自動実行
- 「2年間US$0.00」特典取得を全自動化すると主張
- READMEに「本人開発ではない、学習用」と免責記載あり
- 規約違反・不正利用・詐欺の懸念が大きく、実用は推奨できない

何をするものか

READMEによれば、このツールはGoogle AI Pro（5TB相当）の無料特典を「24か月間US$0.00」で取得する作業を全自動化するものとされています。Windows上でバッチファイルを実行するとローカルサーバーが立ち上がり、ブラウザで管理コンソール（http://127.0.0.1:8765）が開きます。コンソール左側に支払いカード情報を、右側に複数のGoogleアカウントを入力し、ボタン操作で自動実行する構成です。ただし、他者の規約に反してアカウントと決済を自動操作する行為であり、内容面で重大な問題を含みます。

主な機能（READMEの記載）

READMEに記載された自動実行の流れは次の通りです。Step1でGoogleへ自動ログインし、補助メール（復旧用メール）や2FAキーで認証を通過。Step2でカードを登録し12か月の無料パッケージを適用、Step3・Step4でさらに6か月ずつ追加して最大24か月まで積み上げ、Step5でGoogle Playの自動更新をキャンセルしてスクリーンショットで検証するとされています。アカウントは『メール----パスワード----復旧メール----2FAキー』などの形式で一括インポートできると説明されています。

仕組み・アーキテクチャ

ファイル構成はapp.py（FastAPIによるWebサーバー）、automation_engine.py（自動化処理の本体と推測される）、templates/index.html（コンソール画面）、config.json（設定保存）、requirements.txt、および

起動用の2つのバッチファイルから成ります。requirements.txtにはfastapi、uvicorn、playwright、pydanticが挙げられており、uvicornでFastAPIアプリを起動し、Playwrightでヘッドレス/実ブラウザを操作してGoogleのページを自動入力・クリックする設計と考えられます。各ファイルの詳細な実装内容はREADMEからは不明です。

技術構成

主要言語はPythonで、Webフレームワークとしてfastapi（>=0.100.0）、ASGIサーバーとしてuvicorn（>=0.22.0）、ブラウザ自動化にplaywright（>=1.40.0）、データ検証にpydantic（>=2.0.0）を使用しています。実行環境としてWindows 10/11、Google Chrome、Python 3.10以上が前提とされ、米国リージョンのプロキシ利用が推奨されています。ライセンスは不明です。

注目ポイント・重大な注意点

このツールはGoogleの利用規約に明確に違反する自動化・大量アカウント操作・決済操作を行うもので、実行は推奨できません。READMEは「US$0.00でしか課金しない保護機能」を謳いますが、他者のカード情報の取り扱いや、カード情報・アカウント情報を外部へ送信する可能性など、情報窃取やマルウェア的挙動の危険性を排除できません。READMEに記載された卡网（カード販売サイト）やTGグループの存在も、不正アカウント売買に関連する懸念を示唆します。作者自身が『本人開発ではない』と免責している点も含め、信頼性は極めて低いと考えるべきです。

扱う上での姿勢

この種のリポジトリは、技術的にはFastAPIとPlaywrightを組み合わせたブラウザ自動化の構成例として観察できますが、用途は不正利用を前提としています。コードを実行すれば自分や他人の資産・アカウントを危険にさらす可能性が高く、法的・倫理的にも問題があります。学習目的であっても、実際のカード情報やアカウントを投入して動かすべきではありません。

技術構成: Python、FastAPI、Uvicorn、Playwright、Pydantic、HTML（templates/index.html）、Windowsバッチファイル
向いている人: READMEは無料特典を自動取得したいユーザー向けと謳っていますが、実態は規約違反・不正利用を前提としたツールです。安全に利用できる正当な対象ユーザーは存在せず、実行は推奨できません。FastAPIとPlaywrightによるブラウザ自動化の構成を学術的に観察する以上の用途には不適切です。

総合 9. NotRexed/Hookshot

https://github.com/NotRexed/Hookshot

★ 3,235 ／ フォーク 308 ／ 言語 C++ ／ 作成 2026-11-05 ／ GPL-3.0

Switch 2版『ゼルダの伝説 時のオカリナ』を静的再コンパイルし、エミュレータなしでWindows/Linux上でネイティブ実行するプロジェクト

概要

Hookshotは、Nintendo Switch 2版『The Legend of Zelda: Ocarina of Time』のARM64コードをC言語に変換してPC向けにコンパイルする静的リコンパイルプロジェクトです。コンソールのシステムソフトウェアを再実装し、NVN2のグラフィックス呼び出しをVulkan上で再生することで、エミュレータを介さずにWindowsとLinuxで動作させます。リポジトリにはゲーム本体・鍵・実行ファイルは含まれず、ユーザーが自分で所有するゲームデータからPC上でビルドする仕組みです。フレーム生成やアップスケーリング、キー割り当て、RomFS Modの読み込みなど、PCならではの機能も備えています。

要点

- ARM64コードをCに変換してネイティブ実行する静的リコンパイル
- グラフィックスはNVN2呼び出しを記録しVulkanで再生
- ゲーム本体は含まず、自分のゲームデータからPC上でビルド
- Vulkan 1.3とメッシュシェーダー対応GPUが必須
- まだ実験段階で、序盤のコキリの森を中心に動作確認済み

何をするものか

Hookshotは、Switch 2版『時のオカリナ』をWindowsとLinux上でエミュレータなしに動かすためのツール群です。エミュレータが実行時に命令を解釈・変換するのに対し、Hookshotは事前にゲームのコードをCへ翻訳してPC向けの実行ファイルとしてコンパイルします。対応するのは特定のバージョン（program 0400486024DEC000、update v131072、build e0f3ccfa）のみで、他のバージョンは動作しません。リポジトリにはゲーム、鍵、実行ファイルは含まれず、入手方法も説明されていません。生成されるhookshot.exeや生成コード、シェーダーキャッシュ、セーブデータは共有してはいけないと明記されています。

仕組み・アーキテクチャ

READMEによると処理は4段階です。まずnx2/lift.pyがゲームのARM64コードをCに翻訳します。次にnx2/build.pyが、そのCコードとコンソールのシステム役を担うhost/ディレクトリのコードを一緒にコンパイルします。グラフィックスについては、host/nvn_native.cがゲームのNVN2グラフィックス呼び出しを記録し、gpu/vk/がそれをVulkanで再生します。Switch 2のAmpere世代シェーダーはlower_to_maxwell.hでMaxwell形式に下げられ、Edenのシェーダーリコンパイラによって SPIR-V に変換されます。詳細はdocs/development.mdとdocs/design.mdにあるとされています。

主な機能

InsertキーやF1、コントローラーのHomeボタンで開くゲーム内メニューがあり、Graphics、Gameplay、Controlsの3タブで設定できます。品質プリセット（Minimum 4GBからUltra 16GB+まで）、2倍から8倍のフレーム生成、フレームペーシング、Catmull-RomやLanczosによるアップスケーリング、出力解像度、携帯モードプロファイル、ウルトラワイド対応などがあります。ゲームプレイ面ではFPSカウンターやテキスト速度の変更、操作面ではマウスカメラ、感度、キー・ボタン割り当てのカスタマイズが可能です。設定はhookshot_options.iniに保存されます。また、mods フォルダにエミュレータ互換のレイアウトで置いたRomFS Modを読み込めますが、exefsパッチは事前コンパイル方式のため非対応です。

技術構成

主要言語はC++で、リコンパイルされたゲームコードはC、ビルドスクリプトはPythonで書かれています。Windows用のビルダーGUIはbuilder/HookshotBuilder配下にあり、.slnやApp.xamlからC#/.NETのアプリと読み取れます。Linux版ビルダーはPythonスクリプトとAppImageで提供されます。描画はVulkanで、gpu/third_party/suyu配下にsuyu由来のシェーダーリコンパイラ（GLSLやWGSLのバックエンドも含む）が同梱されています。Pythonの依存ライブラリとしてlz4、numpy、capstone、unicorn、zstandardが必要です。コンパイラにはllvm-mingwやclang/lld、ほかにSDL、FFmpeg、Opus、zbic、Vulkan Memory Allocatorなどが使われています。

使い方・導入方法

Windowsでは最新リリースからHookshotBuilder.exeを入手して実行すると、約500MBの残りのツールがダウンロードされます。ゲームの.nupファイルまたはromfsとexefsフォルダを選び、インストール先と言語を指定してビルドすると、CPUやメモリに応じて10〜45分ほどで完了します。LinuxではAppImageに実行権限を付けて起動し、同じ手順でビルドします。完成後はランチャーからPlayで起動でき、新しいリリースが出るとUpdateで差分だけを再ビルドします。開発者はpython -I nx2/make_game.pyでコマンドラインから直接ビルドでき、--checkでバージョン確認だけを行うことも可能です。必要環境は、AVX2対応CPU、RAM 16GB以上、ビルド時の空き容量約60GBです。

注目ポイント・注意点

ステータスは実験段階です。コキリの森での探索、サウンド、コントローラー、セーブは動作します。一方で、初めて訪れる場所でのシェーダー準備によるカクつきや、ハイラル平原での縞状の影、一部でのクラッシュなど未解決の問題があります。GPUはNVIDIAが最もテストされています。AMDは一部で動作報告があるもののRX 6600で既知の問題があり、Intelは報告が少なく黒画面やクラッシュの例が多いとされます。メッシュシェーダー非対応のカード（AMD RX 5000以前など）はサポート外です。ライセンスはGPL-3.0-or-laterで、Eden、suyu、Orchidのコードを含みます。任天堂とは無関係の非公式ファンプロジェクトです。

技術構成: C++、C、Python、C# / .NET 8（Windows用ビルダー）、Vulkan 1.3（メッシュシェーダー）、SPIR-V、Eden / suyu shader recompiler、llvm-mingw、clang / lld、SDL2 / SDL3、FFmpeg、Opus、zbic、Vulkan Memory Allocator、Sirit、capstone、unicorn、lz4、numpy、zstandard、wgpu-native、AppImage
向いている人: 静的リコンパイルやゲーム移植技術、グラフィックスAPIの変換（NVN2からVulkan、Ampere/MaxwellシェーダーからSPIR-V）に関心のあるエンジニアや学生に向いています。対応バージョンのゲームを自分で

所有し、メッシュシェーダー対応GPUを搭載したPCでプレイを試したい人や、テスト報告や修正で貢献したい人にも適しています。

総合 10. govijr/terminal-mac

https://github.com/govijr/terminal-mac

★ 2,401 ／ フォーク 164 ／ 言語 Swift ／ 作成 2026-11-03 ／ MIT

Apple silicon上で仮想macOSを動かし、その画面をターミナルに表示・操作できるようにするSwift製ツール

概要

terminal-macは、Apple Virtualizationで本物のmacOSゲストを起動し、ゲスト内のヘルパーが画面キャプチャと入力イベント送信を担うことで、KittyやiTerm2などのターミナル上で仮想Macを操作できるようにするツールです。主な狙いはAIエージェントのcomputer use（画面操作）用に、ホストとは別のmacOSデスクトップを用意することで、MCPツールやClaude Code向けプラグイン・スキルを同梱しています。ホストの実際のマウスポインタやクリップボードに触れずにエージェントへGUI操作をさせられる点が特徴です。現状はアーリーリリース候補で、クリーンなMac上でのゲスト動作の受け入れ検証はまだ保留中と明記されています。

要点

- Apple Virtualizationで本物のmacOSゲストを起動する
- Ghostty・Kitty・iTerm2のグラフィックス機能で画面をターミナル表示
- MCPツールでエージェントが仮想Macをクリック・入力操作できる
- ホストのポインタやクリップボードに影響しない分離設計
- Apple silicon・macOS 14以降が必須で、現状はアーリーRC

何をするものか

terminal-macは、ホストのMacとは別の仮想macOSデスクトップを用意し、それをターミナルから見たり操作したりするためのツールです。README では「computer-use agents のための別個の macOS デスクトップ」と位置づけられています。ゲストは独自のアプリ、ファイル、アカウント、権限を持つ独立したMacで、ホストで動いているアプリやサインイン済みセッションは引き継ぎません。一方で、VMはホストのCPU、メモリ、ストレージを消費します。対応ターミナル以外では、ネイティブのVMウィンドウとエージェント用CLIを使う形になります。

主な機能

terminal-mac up を実行すると、Appleの復元イメージ（IPSW）の取得とキャッシュ、仮想Macのインストール、起動、ゲストヘルパーの確認までを案内付きで進めます。ビューアは文字入力、左右クリック、ダブル／トリプルクリック、ドラッグ、スクロール、矢印キーなどに対応しています。ターミナルが⌘キーを占有するため、Ctrl+GでSpotlightやMission Controlなどのショートカットパレットを開けます。Ctrl+]でビューアを抜けてもVMは動き続けます。ほかにpause、resume、stop、複数VMの作成、HiDPI表示の切り替え、--offlineでのネットワーク無効化などのライフサイクル管理も備えています。

仕組み・アーキテクチャ

ホストとゲストは virtio-fs の共有フォルダ2つでつながっています。読み書き可能な control はリクエストと返信のJSONやスクリーンショットを置く場所、読み取り専用の tools はヘルパーアプリの置き場です。ゲスト内のTerminalMacGuest.appはScreenCaptureKitで画面を取得し、変化があったときだけ最大30fpsでフレームを送ります。フレームはターミナルの描画サイズに合わせ、lz4で可逆圧縮されます。KittyとGhosttyにはptyではなく共有メモリ経由で渡します。ネットワーク上で待ち受けるリモート操作用のリスナーは持たず、メールボックスはシンボリックリンクの拒否、メッセージサイズの上限、リクエストIDの検証、期限切れ処理などで守られています。ヘルパーは hw.model が VirtualMac で始まる環境でしか起動しません。

技術構成

主要言語はSwiftで、ビルドにはXcode 16以降とSwift 6が必要です。ソースは次のモジュールに分かれています。TerminalMacはCLI、VM、ビューア、エージェントサーバ、ゲストキャプチャを担当します。TerminalMacCoreはメールボックス、モデル、コード署名の識別、ターミナル入力を扱います。TerminalUIはターミナル描画を担当します。ほかにC言語のCPhoneShimがあります。使用しているAppleのフレームワークは、Virtualization framework（VZVirtualMachineViewなど）とScreenCaptureKitです。ホスト実行ファイルにはcom.apple.security.virtualizationエンタイトルメントが必要なため、パッケージ化されたランチャーを使います。テストはswift testで、メールボックスの往復、ファイル境界の防御、入力シーケンスなどを検証します。

使い方・導入方法

ビルド済みのリリースは curl -fsSL https://github.com/govijr/terminal-mac/releases/latest/download/get.sh | bash で導入できます。インストーラはSHA-256を検証したうえで、~/.local/bin にランチャーを置きます。管理者権限は不要です。初回起動ではゲストのmacOSセットアップを手作業で済ませます。その後、ゲスト内で /Volumes/My Shared Files/tools/TerminalMacGuest.app を開き、Screen RecordingとAccessibilityの権限を付与します。エージェント連携は、Claude Codeなら /plugin install terminal-mac --marketplace govijr/terminal-mac で設定できます。その他のMCPクライアントでは terminal-mac mcp --config の出力を設定に貼り付けます。CLIからは action default click 640 400 のような形で、JSONを返すコマンドとしても操作できます。

注目ポイント・注意点

エージェントにGUI操作をさせつつ、ホストのデスクトップを守れる点が最大の魅力です。ただし、既定のNATネットワークではゲストがインターネットやホスト、ローカルネットワークに到達できます。READMEも、悪意あるコードを動かすための堅牢な隔離環境ではないと明記しています。リリースは自己署名でAppleの公証は受けていません。ゲストのセットアップも無人化されていません。snapshot は現状スクリーンショットの別名で、アクセシビリティツリーは取得できません。VMのクローン、ロールバック、tmux経由の表示、VM削除コマンドは未実装です。初回はmacOSイメージの数GBのダウンロードが必要で、ディスクも既定で論理64GiBを使います。

技術構成: Swift 6、Swift Package Manager、Apple Virtualization framework、ScreenCaptureKit、virtio-fs、lz4、C (CPhoneShim)、MCP、Kitty / iTerm2 graphics protocol、Bash（インストール・ビルドスクリプト）
向いている人: Claude Code、Cursor、CodexなどのAIエージェントに、ホストとは別のmacOS環境でGUI操作

（computer use）をさせたい開発者に向いています。ターミナル中心の作業環境で仮想Macを扱いたい人や、Apple VirtualizationやエージェントMCPツールの実装例を学びたいSwiftエンジニアにも参考になります。Apple siliconとmacOS 14以降が前提で、アーリーRC段階のため試験的な利用が中心になります。